JUPITER · SECURITY REVIEW

Jupiter
Lend Referral Program.

A security review of Lend Referral Program.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Jupiter	Lend Referral Program	Jakub Zmyslowski, Zdravko Hristov, Vlad Zotov, Michael Lett	August 24 – September 28, 2026	November 6, 2026

§ 00

Contents.

27 pages: overview, confidence ranking, scope, 16 findings, disclaimer, and about Guardian.

§ 01 **Overview** P. 03

§ 02 **Confidence Ranking** P. 04

§ 03 **Scope & Classifications** P. 05

Files in Scope P. 06

§ 04 **Findings** P. 08

Main Review · 10 findings P. 09

Remediation Review · 6 findings P. 20

§ 05 **Disclaimer** P. 26

§ 06 **About Guardian** P. 27

§ 01

Overview.

Guardian reviewed Lend Referral Program across 2 review rounds. The final report includes 3 Low, 13 Informational findings.

AUDIT FIRM
Guardian

CLIENT
Jupiter

PROJECT
Lend Referral Program

PREPARED BY
Jakub Zmyslowski, Zdravko Hristov, Vlad Zotov, Michael Lett

AUDIT WINDOW
August 24 – September 28, 2026

FINAL REPORT
November 6, 2026

REPOSITORY
`https://github.com/Instadapp/lend-referral`

METHODOLOGY
Smart contract · Manual · Main Review · Remediation Review

MAIN REVIEW COMMIT
`f57d29e08454`

REMEDIATION REVIEW COMMIT
`7be5e230f873`

1.1 Confidence Ranking

5 / 5

Very High Confidence

Guardian assigns a Confidence Ranking of 5 out of 5, combining finding severity, remediation outcome, and the reviewer's assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
Critical	**0**	0	0	0	0
High	**0**	0	0	0	0
Medium	**0**	0	0	0	0
Low	**3**	0	1	0	2
Informational	**13**	0	7	0	6
All severities	**16**	**0**	**8**	**0**	**8**

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence ASSIGNED	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Lend Referral Program in `https://github.com/Instadapp/lend-referral`. The scope includes 2,849 source lines across 31 files.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH	Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.
IMPACT · MEDIUM	Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.
IMPACT · LOW	Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.
LIKELIHOOD · HIGH	Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.
LIKELIHOOD · MEDIUM	Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.
LIKELIHOOD · LOW	Unlikely to ever occur in production.

Files in Scope 1/2.

FILE	NSLOC	TOTAL
MAIN REVIEW		
programs/lending_referral/src/constant.rs	9	22
programs/lending_referral/src/errors.rs	34	35
programs/lending_referral/src/events.rs	101	116
programs/lending_referral/src/helper.rs	33	41
programs/lending_referral/src/invokes/lending.rs	155	179
programs/lending_referral/src/invokes/mod.rs	2	2
programs/lending_referral/src/lib.rs	132	182
programs/lending_referral/src/module/admin.rs	177	216
programs/lending_referral/src/module/mod.rs	4	5
programs/lending_referral/src/module/user.rs	103	128
programs/lending_referral/src/state/context.rs	692	896
programs/lending_referral/src/state/mod.rs	6	7
programs/lending_referral/src/state/seeds.rs	2	2
programs/lending_referral/src/state/state.rs	91	103
programs/lending_referral/src/utils/deposit.rs	238	303
programs/lending_referral/src/utils/mod.rs	2	2
programs/lending_referral/src/utils/withdraw.rs	237	319
REMEDIATION REVIEW		
idl/types/lending_referral.ts	525	4,371
programs/lending_referral/src/constant.rs	2	27
programs/lending_referral/src/errors.rs	9	39
programs/lending_referral/src/events.rs	14	130
programs/lending_referral/src/helper.rs	33	75
programs/lending_referral/src/invokes/lending.rs	0	157
programs/lending_referral/src/lib.rs	15	192
programs/lending_referral/src/module/admin.rs	43	248
programs/lending_referral/src/module/user.rs	8	128
programs/lending_referral/src/state/context.rs	21	870
programs/lending_referral/src/state/state.rs	16	108
programs/lending_referral/src/utils/deposit.rs	49	318
programs/lending_referral/src/utils/withdraw.rs	36	340
ts-sdk/lending_referral/accounts/resolver.ts	3	457
ts-sdk/lending_referral/accounts/types.ts	3	34
ts-sdk/lending_referral/client.ts	9	484
ts-sdk/lending_referral/constants.ts	1	49

Files in Scope 2/2.

FILE	NSLOC	TOTAL
ts-sdk/lending_referral/errors/index.ts	2	107
ts-sdk/lending_referral/instructions/admin.ts	3	132
ts-sdk/lending_referral/instructions/index.ts	10	120
ts-sdk/lending_referral/instructions/params.ts	2	71
ts-sdk/lending_referral/instructions/user.ts	8	174
ts-sdk/lending_referral/math/fees.ts	8	75
ts-sdk/lending_referral/math/shares.ts	4	70
ts-sdk/lending_referral/state/protocol.ts	2	26
ts-sdk/lending_referral/state/wrapper.ts	5	87

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in. Status is where each finding ended up; its own page says how.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
L-01	Protocol claims lose fractional fees	Rounding	LOW	RESOLVED	p. 9
L-02	Fee updates reallocate past yield	Business logic errors	LOW	ACKNOWLEDGED	p. 10
L-03	Withdrawal residuals distort share accounting	Rounding	LOW	RESOLVED	p. 11
I-01	Protocol fee uses inconsistent terminology	Informational	INFORMATIONAL	RESOLVED	p. 13
I-02	Claim authority event omits previous value	Events	INFORMATIONAL	RESOLVED	p. 14
I-03	Declared errors are unreachable	Error	INFORMATIONAL	RESOLVED	p. 15
I-04	LogAccrue is not emitted for 0 fees and omits wrapper	Events	INFORMATIONAL	RESOLVED	p. 16
I-05	Fees may accrue on unrecoverable yield	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 17
I-06	Share redemption uses inconsistent naming	Best Practices	INFORMATIONAL	RESOLVED	p. 18
I-07	Underlying account is not constrained as ATA	Documentation	INFORMATIONAL	RESOLVED	p. 19
REMEDIATION REVIEW					
I-01	`README.md` references outdated fee variable	Out of Scope	INFORMATIONAL	ACKNOWLEDGED	p. 20
I-02	SDK share estimators use an inconsistent share supply	Out of Scope	INFORMATIONAL	ACKNOWLEDGED	p. 21
I-03	Partner fee updates can perform accrual while paused	Unexpected Behavior	INFORMATIONAL	ACKNOWLEDGED	p. 22
I-04	Client error codes do not match the program	Out of Scope	INFORMATIONAL	ACKNOWLEDGED	p. 23
I-05	Fixed share floor may be economically high	Compatibility	INFORMATIONAL	ACKNOWLEDGED	p. 24
I-06	Redeem events omit fractional accounting changes	Events	INFORMATIONAL	ACKNOWLEDGED	p. 25

LOW **L-01** RESOLVED

Protocol claims lose fractional fees

CATEGORY	Rounding
LOCATION	https://github.com/GuardianOrg/mirror-Instadapp-lend-referral-01a01f51/blob/f57d29e08454e9b315b5d25c30ebfbd2f7bc3aaf/programs/lending_referral/src/utils/withdraw.rs#L277-L318
REVIEW	Main Review

DESCRIPTION

Protocol fee balances are stored as scaled `u128` values using `INTERNAL_SCALE`, allowing the protocol to account for fractions smaller than one transferable fToken base unit. However, `claim_protocol_fees` subtracts the entire requested scaled amount before converting it into a transferable `u64` amount using floor division.

For example, with `INTERNAL_SCALE = 1_000_000`, a protocol balance of `1_500_000` represents 1.5 transferable base units. Claiming the entire balance clears all `1_500_000` scaled units from protocol accounting but transfers only one base unit.

The remaining half unit stays in the vault. Because the partner balance is calculated as the vault balance minus the user and protocol allocations, this remainder is implicitly reassigned to the partner bucket, and it may eventually become claimable by the partner after accumulating with other fractional balances.

The protocol can therefore lose up to almost one fToken base unit on every claim.

RECOMMENDATION

Convert the requested amount into raw transferable units first and subtract only `raw_amount * INTERNAL_SCALE` from `protocol_f_token_balance`. This preserves any fractional remainder in the protocol bucket.

RESOLUTION

Status: Resolved.

https://github.com/Instadapp/lend-referral/blob/main/programs/lending_referral/src/utils/withdraw.rs#L277-L323

LOW **L-02** ACKNOWLEDGED

Fee updates reallocate past yield

CATEGORY	Business logic errors
LOCATION	https://github.com/GuardianOrg/mirror-Instadapp-lend-referral-01a01f51/blob/f57d29e08454e9b315b5d25c30ebfbd2f7bc3aaf/programs/lending_referral/src/module/admin.rs#L37-L161
REVIEW	Main Review

DESCRIPTION

Yield fees are accrued when a wrapper interaction calls `accrue`. The accrual covers the entire fToken price increase since `last_f_token_price`, and the current `maintainer_fee_bps` determines how that accumulated fee is divided between the protocol and the partner.

`update_partner_fee` checkpoints the wrapper using the old rates before updating `partner_fee_bps`. In contrast, `update_protocol_yield_fee` immediately changes `maintainer_fee_bps` without checkpointing any wrappers.

As a result, the new maintenance fee is applied to yield earned before the update, incorrectly shifting pending fees between the protocol and the partner.

RECOMMENDATION

Checkpoint each wrapper's pending yield using the old maintenance fee before applying the new rate.

RESOLUTION

Status: Acknowledged.

LOW L-03

RESOLVED

Withdrawal residuals distort share accounting

CATEGORY	Rounding
LOCATION	https://github.com/GuardianOrg/mirror-Instadapp-lend-referral-01a01f51/blob/4287f859bd79ffd38f66eec88689ada15649a423/programs/lending_referral/src/utils/withdraw.rs
REVIEW	Main Review

DESCRIPTION

`withdraw_to_f_token()` and `redeem_to_underlying()` calculate the withdrawing holder's proportional `f_scaled`, but transfer only `floor(f_scaled / INTERNAL_SCALE)` raw fTokens and deduct only the transferred amount multiplied by `INTERNAL_SCALE`. The remaining `f_scaled % INTERNAL_SCALE` stays in `user_f_token_balance` even though the associated xTokens were burned.

This residual increases the fToken value of the remaining xTokens. Repeated small deposits and withdrawals can compound the increase, causing later depositors to receive fewer xTokens. An independently calculated `min_x_out` protects depositors, but unprotected deposits or minimums derived from an already-distorted state do not.

When the final holder exits, the same behavior leaves `user_f_token_balance % INTERNAL_SCALE` despite `total_x_shares` being zero. The PoC ends with `998,515` scaled user units and one raw fToken remaining in the wrapper. This value is stranded or inherited by the next depositor, potentially seeding the same distortion again.

RECOMMENDATION

If withdrawal residuals may belong to the partner, subtract the complete `f_scaled` from `user_f_token_balance`. The untransferred remainder will then enter the partner's implicit balance and the final withdrawal will clear the user balance.

Otherwise, store the remainder in a separate scaled reserve and update `claim_partner_fees()`.

PROOF OF CONCEPT https://github.com/GuardianOrg/mirror-Instadapp-lend-referral-01a01f51/blob/4287f859bd79ffd38f66eec88689ada15649a423/tests/tests/optimized_rounding_extraction.rs

RESOLUTION

Status: Resolved.

L-03 Withdrawal residuals distort share accounting CONTINUED

https://github.com/Instadapp/lend-referral/blob/fix/audit_fixes/programs/lending_referral/src/utils/withdraw.rs#L18-L143

INFORMATIONAL **I-01** RESOLVED

Protocol fee uses inconsistent terminology

CATEGORY	Informational
LOCATION	https://github.com/GuardianOrg/mirror-Instadapp-lend-referral-01a01f51/blob/f57d29e08454e9b315b5d25c30ebfbd2f7bc3aaf/programs/lending_referral/src/module/admin.rs#L37-L54
REVIEW	Main Review

DESCRIPTION

The protocol's share of accrued yield fees is inconsistently described as both a "yield fee" and a "maintenance fee." `update_protocol_yield_fee()` updates `ProtocolConfig.maintainer_fee_bps`, validates it against `MAX_MAINTENANCE_FEE_BPS`, and may return `ProtocolMaintenanceFeeExceedsMax`, while the emitted event is named `LogUpdateProtocolYieldFee`. This inconsistent terminology extends across state, events, errors, tests, and the SDK, making it unclear whether the names represent separate fees.

RECOMMENDATION

Adopt one term throughout the program, IDL, SDK, events, errors, and tests. Since the value represents the protocol's share of collected yield fees, consider naming it `protocol_fee_share_bps` and renaming related identifiers accordingly.

RESOLUTION

Status: Resolved.

https://github.com/Instadapp/lend-referral/blob/fix/audit_fixes/programs/lending_referral/src/module/admin.rs#L14-L54

INFORMATIONAL I-02 RESOLVED

Claim authority event omits previous value

CATEGORY	Events
LOCATION	https://github.com/GuardianOrg/mirror-Instadapp-lend-referral-01a01f51/blob/f57d29e08454e9b315b5d25c30ebfbd2f7bc3aaf/programs/lending_referral/src/events.rs#L55-L59
REVIEW	Main Review

DESCRIPTION

`LogUpdateClaimAuthority` only records `new_claim_authority`, while comparable administrative update events record both the old and new values. Consequently, event consumers cannot reconstruct the complete authority transition from the event alone.

RECOMMENDATION

Add `old_claim_authority` to `LogUpdateClaimAuthority` and capture it before updating `WrapperConfig.claim_authority` in `update_claim_authority()`.

RESOLUTION

Status: Resolved.

https://github.com/Instadapp/lend-referral/blob/fix/audit_fixes/programs/lending_referral/src/events.rs#L55-L60

INFORMATIONAL I-03 RESOLVED

Declared errors are unreachable

CATEGORY	Error
LOCATION	https://github.com/Instadapp/lend-referral/blob/f57d29e08454e9b315b5d25c30ebfbd2f7bc3aaf/programs/lending_referral/src/state/state.rs#L51-L53 https://github.com/Instadapp/lend-referral/blob/f57d29e08454e9b315b5d25c30ebfbd2f7bc3aaf/crates/library/src/math/safe_math.rs#L51-L60
REVIEW	Main Review

DESCRIPTION

The program declares `WrapperVaultBalanceUnderrun` and `MathOverflow` errors, but neither is returned as intended. If tracked balances exceed the vault, `safe_sub` returns the shared library's `LibraryMathError`, not `WrapperVaultBalanceUnderrun`. `MathOverflow` is unreachable because the divided result cannot exceed the original `u64` vault balance. As a result, the IDL and SDK expose error codes that clients will never receive, leading to misleading diagnostics and error handling.

RECOMMENDATION

Return `WrapperVaultBalanceUnderrun` explicitly when tracked balances exceed the vault, and remove the unreachable `MathOverflow` error.

RESOLUTION

Status: Resolved.

https://github.com/Instadapp/lend-referral/blob/fix/audit_fixes/programs/lending_referral/src/state/state.rs#L46-L56

INFORMATIONAL I-04 RESOLVED

LogAccrue is not emitted for 0 fees and omits wrapper

CATEGORY	Events
LOCATION	https://github.com/GuardianOrg/mirror-Instadapp-lend-referral-01a01f51/blob/f57d29e08454e9b315b5d25c30ebfbd2f7bc3aaf/programs/lending_referral/src/utils/deposit.rs#L44-L72
REVIEW	Main Review

DESCRIPTION

When the lending exchange price increases, `accrue()` updates `last_f_token_price` before checking whether `total_delta_scaled` is zero. If it is zero, the function returns without emitting `LogAccrue`.

This occurs when the wrapper has no user deposits, `partner_fee_bps` is disabled, or the fee calculated by `calc_total_delta()` rounds down to zero. Consequently, `accrue()` can successfully advance the stored price checkpoint without producing the event intended to report that accrual. Off-chain indexers and monitoring systems relying on `LogAccrue` cannot distinguish these successful zero-fee accruals from cases where `accrue()` was never executed, potentially leaving their observed price checkpoint stale.

Additionally, when `LogAccrue` is emitted, it does not include the affected wrapper_config. Since `accrue` can operate on any wrapper event consumers cannot attribute an emitted accrual to a wrapper without fetching and decoding the complete transaction.

RECOMMENDATION

Either emit `LogAccrue` after updating `last_f_token_price`, including when `total_delta_scaled` is zero, or document this event reports only fee accruals.

Also consider including the wrapper config in the emitted data.

RESOLUTION

Status: Resolved.

https://github.com/Instadapp/lend-referral/blob/fix/audit_fixes/programs/lending_referral/src/utils/deposit.rs#L19-L72

INFORMATIONAL I-05 ACKNOWLEDGED

Fees may accrue on unrecoverable yield

CATEGORY	Informational
LOCATION	https://github.com/GuardianOrg/mirror-Instadapp-lend-referral-01a01f51/blob/f57d29e08454e9b315b5d25c30ebfbd2f7bc3aaf/programs/lending_referral/src/utils/deposit.rs#L37-L42
REVIEW	Main Review

DESCRIPTION

`accrue()` treats all `token_exchange_price` growth as realizable yield. If Liquidity continues accruing unrecoverable debt at face value, part of this nominal yield is allocated as protocol and partner `fToken` fees. Liquidity does not socialize bad debt through a supplier write-down; early redeemers can consume the remaining liquidity, leaving the shortfall to later suppliers. Fee recipients who redeem first may therefore exit ahead of depositors. This requires an upstream bad-debt event.

RECOMMENDATION

Bad debt occurs at Liquidity's level, so no changes are needed in the current program, but be aware of this possibility.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-06 RESOLVED

Share redemption uses inconsistent naming

CATEGORY	Best Practices
LOCATION	https://github.com/GuardianOrg/mirror-Instadapp-lend-referral-01a01f51/blob/f57d29e08454e9b315b5d25c30ebfbd2f7bc3aaf/programs/lending_referral/src/lib.rs#L115-L124
REVIEW	Main Review

DESCRIPTION

`withdraw_to_f_token()` and `withdraw_to_f_token_with_min_out()` burn an exact amount of `xToken` shares and return a variable amount of `fToken`s. This matches `redeem_to_underlying()` semantics, whereas `withdraw_to_underlying()` specifies the exact asset output.

RECOMMENDATION

Rename them to `redeem_to_f_token()` and `redeem_to_f_token_with_min_out()`, updating the SDK, IDL and all code these actions depend on.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-07 RESOLVED

Underlying account is not constrained as ATA

CATEGORY	Documentation
LOCATION	https://github.com/GuardianOrg/mirror-Instadapp-lend-referral-01a01f51/blob/f57d29e08454e9b315b5d25c30ebfbd2f7bc3aaf/programs/lending_referral/src/state/context.rs#L762-L769
REVIEW	Main Review

DESCRIPTION

`UnderlyingOp` documents `user_underlying_account` as the user's underlying ATA, but its constraints permit any token account with the expected mint and signer authority.

RECOMMENDATION

Either update the comment to describe a signer-owned token account or enforce ATA derivation with `associated_token::mint` and `associated_token::authority`.

RESOLUTION

Status: Resolved.

https://github.com/Instadapp/lend-referral/blob/fix/audit_fixes/programs/lending_referral/src/state/context.rs#L762-L770

INFORMATIONAL **I-01** ACKNOWLEDGED

README.md references outdated fee variable

CATEGORY	Out of Scope
LOCATION	https://github.com/GuardianOrg/mirror-Instadapp-lend-referral-01a01f51/blob/4818c228e05f14898668d95f32ef165e537430a0/README.md#L50
REVIEW	Remediation Review

DESCRIPTION

The protocol fee variable was renamed from `maintainer_fee_bps` to `protocol_fee_share_bps`, but `README.md` still references the old name. This may confuse integrators looking for a field that no longer exists.

RECOMMENDATION

Update `README.md` to reference `protocol_fee_share_bps`.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-02 ACKNOWLEDGED

SDK share estimators use an inconsistent share supply

CATEGORY	Out of Scope
LOCATION	https://github.com/GuardianOrg/mirror-Instadapp-lend-referral-01a01f51/blob/7be5e230f873fb6edf68ecdd773dc2cadbf682e1/ts-sdk/lending_referral/client.ts#L90-L92 https://github.com/GuardianOrg/mirror-Instadapp-lend-referral-01a01f51/blob/7be5e230f873fb6edf68ecdd773dc2cadbf682e1/ts-sdk/lending_referral/client.ts#L151-L153
REVIEW	Remediation Review

DESCRIPTION

The program performs share accounting using `WrapperConfig.total_x_shares`, which is intentionally not equivalent to the SPL `xToken` mint supply. During the first deposit, the program adds the complete `f_amount` to `total_x_shares` but only mints `f_amount - MIN_LOCKED_SHARES` transferable shares. Furthermore, holders can burn their `xToken`s directly through the SPL Token Program without updating `total_x_shares`.

However, the SDK's `depositFToken()` and `redeemToFToken()` functions read the SPL mint supply and pass it as `xSupply` to `estimateXTokensOut()` and `estimateFTokensOut()`. The SDK's `WrapperConfig` type does not expose `total_x_shares`, and `estimateXTokensOut()` additionally treats zero mint supply as a one-to-one initial deposit.

Consequently, SDK estimates and generated slippage bounds can diverge from the on-chain calculations.

RECOMMENDATION

Make the SDK support the newest floor addition and also use the wrapper config supply values instead of reading the raw token amounts.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-03 ACKNOWLEDGED

Partner fee updates can perform accrual while paused

CATEGORY	Unexpected Behavior
LOCATION	https://github.com/GuardianOrg/mirror-Instadapp-lend-referral-01a01f51/blob/7be5e230f873fb6edf68ecdd773dc2cadbf682e1/programs/lending_referral/src/state/context.rs#L299-L304 https://github.com/GuardianOrg/mirror-Instadapp-lend-referral-01a01f51/blob/7be5e230f873fb6edf68ecdd773dc2cadbf682e1/programs/lending_referral/src/module/admin.rs#L173-L181
REVIEW	Remediation Review

DESCRIPTION

The standalone `accrue()` instruction rejects execution when `protocol_config.paused` is set. However, the `UpdatePartnerFee` context does not enforce this pause condition, and `update_partner_fee()` directly calls the internal `accrue()` function before changing the partner fee.

Consequently, an authorized wrapper administrator can refresh the lending rate and update referral accounting while the protocol is paused.

RECOMMENDATION

Document that administrative operations such as `update_partner_fee()` may continue accruing while paused.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-04 ACKNOWLEDGED

Client error codes do not match the program

CATEGORY	Out of Scope
LOCATION	`ts-sdk/lending_referral/errors/index.ts:57`
REVIEW	Remediation Review

DESCRIPTION

The program defines its custom errors with an offset of `7000`, while the SDK's `ERROR_MAP` still uses the previous `6000`-based codes. Consequently, `parseProgramError` does not recognize the program's current custom errors and returns `Unknown`.

For example, the program returns `WrapperVaultBalanceUnderrun` with code `7011`, but the SDK maps that error to `6011`.
Also, the SDK retains the removed `MathOverflowError` and outdated mappings for subsequent errors.

Clients relying on these definitions receive misleading diagnostics, and application logic that handles specific program errors may fail to execute.

RECOMMENDATION

Update the SDK's error classes and mappings to match the program's current error codes, and remove `MathOverflowError`.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-05

ACKNOWLEDGED

Fixed share floor may be economically high

CATEGORY	Compatibility
LOCATION	https://github.com/GuardianOrg/mirror-Instadapp-lend-referral-01a01f51/blob/4818c228e05f14898668d95f32ef165e537430a0/programs/lending_referral/src/constant.rs#L27
REVIEW	Remediation Review

DESCRIPTION

The program is expected to support tokens with 2–9 decimals, but `MIN_LOCKED_SHARES` is fixed at 1,000 raw units. For a two-decimal token, the first successful deposit permanently locks 10 whole fTokens, which may represent a disproportionate economic loss for the initial depositor.

RECOMMENDATION

Scale `MIN_LOCKED_SHARES` according to the token's decimals or configure it per wrapper so the locked economic value remains negligible across supported assets.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-06 ACKNOWLEDGED

Redeem events omit fractional accounting changes

CATEGORY	Events
LOCATION	https://github.com/GuardianOrg/mirror-Instadapp-lend-referral-01a01f51/blob/7be5e230f873fb6edf68ecdd773dc2cadbf682e1/programs/lending_referral/src/utils/withdraw.rs#L39-L74 https://github.com/GuardianOrg/mirror-Instadapp-lend-referral-01a01f51/blob/7be5e230f873fb6edf68ecdd773dc2cadbf682e1/programs/lending_referral/src/utils/withdraw.rs#L97-L144
REVIEW	Remediation Review

DESCRIPTION

`redeem_to_f_token()` and `redeem_to_underlying()` subtract the full scaled entitlement, `f_scaled`, from `total_user_f_token_balance`, while their events report only the floored transferable fToken amount. Consequently, the fractional value reassigned to the partner bucket is not represented in the events, preventing them from independently describing the complete accounting transition.

RECOMMENDATION

Consider adding the scaled user-balance reduction, or the fractional remainder, to both events.

RESOLUTION

Status: Acknowledged.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

$45B+
DIGITAL ASSETS UNDER REVIEW

200+
ENGAGEMENTS COMPLETED

500+
CRITICAL & HIGH FINDINGS REPORTED

Reach us at audits@guardianaudits.com.